

```

1  #include "FibonacciSequenceIterator.h"
2
3  // Constructor - Initializes the iterator by setting
   fSequence to the provided FibonacciSequence pointer and
   fIndex to the specified starting index.
4  FibonacciSequenceIterator::FibonacciSequenceIterator(
   FibonacciSequence* aSequence, uint64_t aStart) noexcept
5      : fSequence(aSequence), fIndex(aStart)
6  {
7  }
8
9  // Operator* - Returns a constant reference to the current
   Fibonacci number by dereferencing the underlying
   FibonacciSequence.
10 const uint64_t& FibonacciSequenceIterator::operator*()
   const noexcept {
11     return **fSequence;
12 }
13
14 // Prefix Increment operator - Advances the underlying
   FibonacciSequence and increments the iterator index.
15 FibonacciSequenceIterator& FibonacciSequenceIterator::
   operator++() noexcept {
16     ++(*fSequence);           // Advance the
   Fibonacci sequence
17     ++fIndex;                 // Increment
   the index
18     return *this;            // Return the
   updated iterator
19 }
20
21 // Postfix Increment operator - Saves the current iterator
   state, advances the iterator using the prefix operator,
   and returns the saved state.
22 FibonacciSequenceIterator FibonacciSequenceIterator::
   operator++(int) noexcept {
23     FibonacciSequenceIterator temp = *this; // Save
   current iterator state
24     ++(*this);           // Advance to
   the next state
25     return temp;         // Return the
   old state
26 }
27

```

```
28 // Equality operator - Compares the iterator by checking
   // if both fSequence pointers and fIndex values are equal.
29 bool FibonacciSequenceIterator::operator==(const
   FibonacciSequenceIterator& aOther) const noexcept {
30     return (fSequence == aOther.fSequence) && (fIndex ==
   aOther.fIndex);
31 }
32
33 // begin - Resets the underlying FibonacciSequence to its
   // initial state and returns a new iterator with index 0.
34 FibonacciSequenceIterator FibonacciSequenceIterator::begin
   () const noexcept {
35     fSequence->begin();
36     return FibonacciSequenceIterator(fSequence, 0);
37 }
38
39 // end - Returns a new iterator representing the end of
   // the sequence by setting fIndex to MAX_FIBONACCI.
40 FibonacciSequenceIterator FibonacciSequenceIterator::end
   () const noexcept {
41     return FibonacciSequenceIterator(fSequence,
   MAX_FIBONACCI);
42 }
43
```